

Build Your First AI-Powered Application

From LLM APIs to RAG

A hands-on session · Python · everything we use is free

What we're building

A question-answering app

It reads *your* notes and answers from them, cites which file each answer came from, and says “I don't know” when the answer isn't there.

A mental model

You'll be able to look at any AI product — a support bot, Copilot, Perplexity — and correctly guess how it was built.

Something to take further

Six project briefs you can turn into a portfolio piece, a hackathon entry, or a final-year project.

About 150 lines of Python. No maths, no GPUs, no model training.

What a language model actually is

A function that takes text and predicts what text comes next — one piece at a time, until it decides to stop.

Everything else — chat, agents, coding assistants, RAG — is engineering wrapped around that one function.

- No memory between calls
- No internet, no database, no clock, unless you give it one
- Knowledge frozen at a training cutoff date
- Probabilistic — the same prompt can give different answers

"The capital of France is" → " **Paris**"

"...France is Paris" → "."

"...is Paris." → **stop**

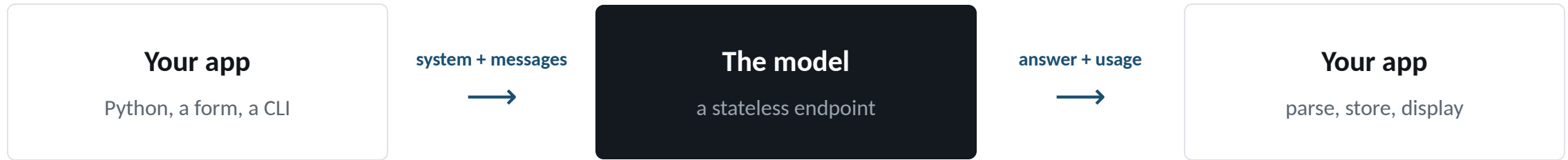
When a model makes something up, it hasn't malfunctioned. It produced plausible next text. **Plausible isn't true.**

Our job today is feeding it the right context so that plausible becomes true.

Words you'll hear today

TERM	WHAT IT MEANS
Token	Roughly four characters. You are billed, and rate-limited, per token — in <i>and</i> out.
Context window	Everything the model can see in one call. Its entire working memory.
Temperature	0 is repeatable and predictable. 1 takes creative risks. Extraction and RAG want low.
System prompt	Standing instructions sent on every call. Most of your engineering lives here.
Stateless	The API remembers nothing. “Memory” is your app re-sending the conversation.
Grounding	Putting real, retrieved facts in the prompt so the answer is checkable.

The shape of every AI application



Every question you'll ever have is about the left box: **what goes into the prompt, and what do I do with what comes back?**

The interesting engineering is never inside the model.

Setup

```
python --version           # 3.10 or newer
python -m venv .venv
source .venv/bin/activate   # Windows: .venv\Scripts\activate

pip install openai sentence-transformers numpy streamlit

export LLM_API_KEY="AIza..." # Windows: setx LLM_API_KEY "AIza..."

mkdir notes                 # put 3-5 .txt files in here
```

Free API key, 60 seconds

aistudio.google.com/apikey

A Google account. No credit card.

Why `openai`?

The OpenAI request format became the standard shape for LLM APIs, so nearly every provider now speaks it. It's a universal client, not a bill.

Never put a key in your code. `.gitignore` your `.env` before the first commit.

Three free options. Same code.

```
BASE_URL = "https://generativelanguage.googleapis.com/v1beta/openai/"
MODEL    = "gemini-2.5-flash"

# BASE_URL = "https://api.groq.com/openai/v1"
# MODEL    = "llama-3.3-70b-versatile"

# BASE_URL = "http://localhost:11434/v1"
# MODEL    = "llama3.2"

client = OpenAI(api_key=os.environ["LLM_API_KEY"], base_url=BASE_URL)
```

OPTION	FREE TIER
Google AI Studio Gemini Flash	No card, no expiry. ~15 requests/min, ~1,500/day
Groq open models	No card. ~30 requests/min. Fastest of the three
Ollama on your laptop	Unlimited, offline, no key. Needs ~8 GB RAM

Two lines. That's the whole difference between providers.

Anatomy of a request

```
resp = client.chat.completions.create(
    model=MODEL,
    max_tokens=300,
    temperature=0.2,
    messages=[
        {"role": "system", "content": "You are a concise tutor."},
        {"role": "user", "content": "What is a pointer?"},
        {"role": "assistant", "content": "A variable holding an address."},
        {"role": "user", "content": "Show me one in C."},
    ],
)

print(resp.choices[0].message.content)
```

FIELD	WHY IT MATTERS
model	Small and fast, or large and slow. Start small.
max_tokens	Caps the answer. It truncates — it doesn't summarise.
system	Persona, rules, output format. Always first.
messages	The full transcript. You maintain this list.
choices[0]	The API can return several candidates. You asked for one.

Lab 1 — your first call

```
import os
from openai import OpenAI

BASE_URL = "https://generativelanguage.googleapis.com/v1beta/openai/"
MODEL = "gemini-2.5-flash"

client = OpenAI(api_key=os.environ["LLM_API_KEY"], base_url=BASE_URL)

resp = client.chat.completions.create(
    model=MODEL,
    max_tokens=300,
    temperature=0.2,
    messages=[
        {"role": "system", "content": "You are a concise teaching "
                                       "assistant. Under 60 words. No preamble."},
        {"role": "user", "content": "Explain recursion to a "
                                     "first-year student."},
    ],
)

print(resp.choices[0].message.content)
print("in:", resp.usage.prompt_tokens,
      "out:", resp.usage.completion_tokens)
```

Then try these

- 1 Delete the system message. How does the answer change?
- 2 Run 3× at `temperature=1.0`, then 3× at `0`.
- 3 Set `max_tokens=20`. Watch what happens.

Lab 1b — giving it a memory

```
SYSTEM = {"role": "system",
          "content": "You are a patient study buddy."}

history = []                                # this list IS the memory

while True:
    q = input("\nYou: ")
    if q.lower() in {"quit", "exit"}:
        break

    history.append({"role": "user", "content": q})

    resp = client.chat.completions.create(
        model=MODEL, max_tokens=500,
        messages=[SYSTEM] + history,
    )
    answer = resp.choices[0].message.content
    history.append({"role": "assistant", "content": answer})
    print("Bot:", answer)
```

There is no memory on the server. You are re-sending the entire transcript on every single call.

Try it

Ask “what is a stack?”, then just “give me an example”. It works.

Now comment out the two `history.append` lines and ask the same follow-up.

Turn 20 re-sends turns 1–19. Long chats get expensive fast.

When it breaks

ERROR	WHAT'S ACTUALLY WRONG	FIX
<code>KeyError: 'LLM_API_KEY'</code>	Set in a different terminal, or your IDE didn't inherit it	Re-export in the window you're running from
<code>401</code> API key not valid	Usually a trailing space from the copy-paste	Check <code>len(key)</code>
<code>404</code> model not found	Model names change every few months	Check the current list, change one line
<code>429</code> rate limit	Free tier is around 15 requests a minute	Wait 60s. Don't panic-rerun
Answer stops mid-sentence	<code>max_tokens</code> truncates, it doesn't shorten	Raise it, or ask for brevity in the prompt
<code>ModuleNotFoundError</code>	Installed into a different Python than you're running	<code>which python</code> — is the venv active?

When an answer is wrong, print the exact prompt you sent before you blame the model. Most of the time it isn't what you thought it was.

A prompt is a program.

Next: turning a chatbot into a component your code can actually consume.

Four things every good prompt has

Role

Who is answering, and for whom. *“You are a teaching assistant for a first-year data structures course.”*

Context

The facts it may use. Retrieved documents, the user's data, today's date. Wrap it in tags so the model can tell data from instructions.

Task

One clear instruction, with the edge cases named. *“If the context doesn't answer it, say so — don't guess.”*

Format

Exactly what comes back. A schema, a word limit, an example. This is the difference between a demo and a component.

Put long context **before** the question. Models attend best to the start and the end of a prompt.

One worked example beats three paragraphs of description. Two examples beat one.

Same task, two prompts

Weak

```
"Summarise these meeting notes."
```

Three friendly paragraphs. A different shape every run. An opening line of “Sure! Here's a summary”. Nothing your code can parse.

Strong

```
You are a project manager's assistant.
```

```
From the meeting notes in <notes> tags, extract every  
commitment someone made.
```

```
Rules:
```

- One entry per commitment. Never merge two.
- owner must be a name that appears in the notes,
or "unassigned". Never invent a person.
- due: ISO date if stated, else null.
- If there are no commitments, return an empty list.

```
Return JSON:
```

```
{"action_items": [{"task": str, "owner": str,  
  "due": str|null, "priority": "high"|"medium"|"low"}]}
```

Every line in the strong prompt is there because something went wrong once. **Prompts grow like test suites — one bug at a time.**

Lab 2 — notes to structured data

```
import json

SYSTEM = """You extract commitments from meeting notes.
Return ONLY valid JSON matching this shape:
{"action_items":[{"task":str,"owner":str,"due":str|null,
                  "priority":"high"|"medium"|"low"}]}
Never invent an owner. Use "unassigned" if unclear.
If there are no commitments, return an empty list."""

notes = open("notes/standup.txt").read()

resp = client.chat.completions.create(
    model=MODEL, max_tokens=800, temperature=0,
    response_format={"type": "json_object"},
    messages=[
        {"role": "system", "content": SYSTEM},
        {"role": "user", "content": f"<notes>\n{notes}\n</notes>"},
    ],
)

data = json.loads(resp.choices[0].message.content)
for it in data["action_items"]:
    print(f'[{it["priority"]:>6}] {it["owner"]:<10} {it["task"]}')

```

Why this matters

Prose needs a human reader. JSON means your program can read it — store it, validate it, branch on it, test it.

JSON mode

`response_format` makes the provider constrain the output so it has to parse. Still wrap `json.loads` in a try/except.

Add `blocked_by` to the schema and rerun. You changed the spec, not the parser.

**The model is capable,
but it knows nothing
about your world.**

Here is the standard fix.

Three questions it cannot answer

Too recent

“What changed in the exam policy last month?”

Training data is frozen at a cutoff. Anything after it doesn't exist for the model.

Too private

“What does our lab handbook say about resubmissions?”

Your notes and documents were never in the training data — and you wouldn't want them to be.

Too specific

“Which clause covers late submission, and where?”

Even if it half-remembers, it can't cite. An answer you can't verify is an answer you can't use.

Ask any of these and you won't get an error. You'll get a confident, well-written, invented answer — which is far more dangerous.

Three ways to teach a model your world

APPROACH	HOW IT WORKS	GOOD WHEN	THE COST
Stuff the prompt	Put the whole document into every request	One small, fixed document	You pay for every token every call; breaks past the context window
Fine-tuning	Keep training the model's weights on your examples	Teaching a style or a format — not facts	Thousands of examples, GPUs, ML skill. Facts go stale the day you finish, and it still hallucinates
RAG retrieval-augmented generation	Search your data first, paste only the top few relevant chunks into the prompt	Facts that change, are private, or are large	You build a search layer. Updating means re-indexing one file. Answers are citable

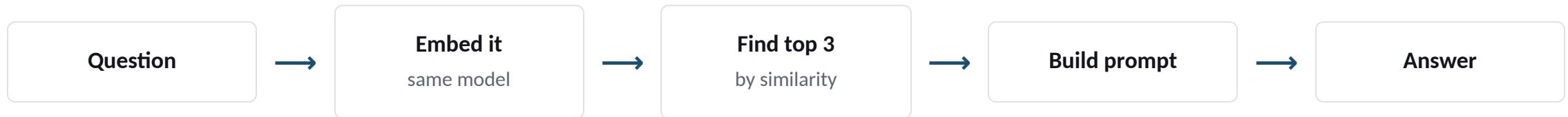
Fine-tuning teaches the model *how to behave*. RAG teaches it *what is true right now*.

RAG has two phases

INDEXING — ONCE PER DOCUMENT



QUERYING — EVERY QUESTION



No training. No GPU. No changes to the model. **RAG is a search engine bolted onto a prompt template.**

Embeddings: meaning as coordinates

An embedding model turns text into a list of numbers, positioned so that texts with similar *meaning* land near each other.

"how do I submit late work?" → [0.12, -0.44, ...]

"late submission policy" → [0.13, -0.41, ...] **close**

"campus parking rules" → [-0.71, 0.08, ...] **far**

Those two phrases share **no keywords**. Ctrl-F finds nothing. Semantic search finds it.

```
from sentence_transformers import SentenceTransformer

embedder = SentenceTransformer("all-MiniLM-L6-v2")
v = embedder.encode(["late submission policy"],
                    normalize_embeddings=True)

print(v.shape)      # (1, 384)
```

This runs on your own CPU. Free, no key, no rate limit.

Two rules

Same embedding model for documents and questions. Re-embed everything if you ever switch models — vectors are not portable.

Similarity is one line of maths

```
# normalize_embeddings=True made every vector length 1,  
# so the dot product IS the cosine similarity.  
  
scores = vectors @ query_vector      # every chunk at once  
top_k   = np.argsort(scores)[::-1][:3]
```

That is your entire vector database. A NumPy array and one matrix multiply. Reach for FAISS or Chroma at around 100,000 chunks — not before.

SCORE	USUALLY MEANS
> 0.75	Near-restatement. Great hit
0.45 - 0.75	Related. Most useful hits live here
0.25 - 0.45	Loosely on topic. Often noise
< 0.25	Unrelated. Drop it

Always returning the top three means a nonsense question still gets three irrelevant chunks — and invites a hallucination. **Set a floor. Let it return nothing.**

Chunking is where most projects go wrong

Too big

One vector has to represent ten different ideas, so it represents none of them well. Retrieval goes vague and you burn tokens.

Too small

You retrieve a fact with no context around it. *“This is not permitted.”* — what isn't? The model has to guess, and it will.

About right

100–250 words with roughly 20% overlap, split on headings and paragraphs before you split on length.

```
def chunk(text, size=180, overlap=40):  
    words, step = text.split(), size - overlap  
    return [" ".join(words[i:i + size])  
            for i in range(0, max(len(words), 1), step)]
```

When your RAG app is bad, change this first — not the model, not the prompt. Print five chunks and read them out loud. If one doesn't make sense to you on its own, it won't make sense to the model either.

What we're building

A command-line assistant that answers questions **only** from the files in your `notes/` folder, cites which file each claim came from, and says “*Not in my notes*” when it doesn't know.

- 1 **Load** — read every file in `notes/`
- 2 **Chunk** — slice into overlapping passages
- 3 **Embed** — one vector per chunk, in a NumPy array
- 4 **Retrieve** — embed the question, take the closest three
- 5 **Generate** — paste those three into a strict prompt

```
ai-workshop/  
  notes/  
    lecture01.txt  
    handbook.txt  
    syllabus.txt  
  rag.py  
  app.py
```

One file, top to bottom. No framework. Once you've done it by hand, LangChain stops looking like magic — it does exactly these five steps.

Steps 1 and 2 — load and chunk

```
import os, glob, numpy as np
from openai import OpenAI
from sentence_transformers import SentenceTransformer

BASE_URL = "https://generativelanguage.googleapis.com/v1beta/openai/"
MODEL = "gemini-2.5-flash"

client = OpenAI(api_key=os.environ["LLM_API_KEY"], base_url=BASE_URL)
embedder = SentenceTransformer("all-MiniLM-L6-v2")

# ----- 1. LOAD -----
docs = []
for path in sorted(glob.glob("notes/*.txt")):
    with open(path, encoding="utf-8") as f:
        docs.append((os.path.basename(path), f.read()))

# ----- 2. CHUNK -----
chunks = [{"source": name, "text": c}
           for name, text in docs for c in chunk(text)]

print(f"{len(docs)} docs -> {len(chunks)} chunks")
```

Keep the filename

That's what lets us cite later. Metadata is not optional.

Then read a chunk

```
print(chunks[5]["text"])
```

Does it stand alone? Ten seconds here saves an hour later.

Step 3 — build the index

```
# ----- 3. EMBED -----  
vectors = embedder.encode(  
    [c["text"] for c in chunks],  
    normalize_embeddings=True,  
    show_progress_bar=True,  
)  
  
print(vectors.shape)      # (37, 384)
```

```
np.save("index.npy", vectors)      # cache it
```

That array is your vector database.

37 chunks, 384 numbers each. That's genuinely all Pinecone and Chroma are, plus indexing tricks that matter at millions of rows.

Nothing here touches the network

The embedding model ran on your CPU. No key, no quota, no cost. Only step 5 spends any of your free daily requests — so iterate on chunking and retrieval freely.

Step 4 — retrieve

```
# ----- 4. RETRIEVE -----  
def retrieve(question, k=3, floor=0.25):  
    qv      = embedder.encode([question], normalize_embeddings=True)[0]  
    scores = vectors @ qv  
    order  = np.argsort(scores)[::-1][:k]  
    return [(chunks[i], float(scores[i]))  
            for i in order if scores[i] >= floor]  
  
for hit, score in retrieve("what is the late submission rule?"):  
    print(f"{score:.2f} {hit['source']} {hit['text'][:80]}...")
```

Test retrieval **before** you wire up the model. If the right paragraph isn't in this list, no prompt will save the answer.

Three things to try

Ask using completely different words than the document uses.

Ask something unrelated — you should get zero hits.

Set `k=8`. Better answers, or just more noise?

Step 5 — augment and generate

```
SYSTEM = """You answer questions about the user's study notes.

Rules:
- Use ONLY the text inside <context>. Never use outside knowledge.
- After each claim, cite the source file, e.g. [syllabus.txt].
- If <context> does not contain the answer, reply exactly:
  "Not in my notes." Do not guess.
- Be concise: 3 sentences maximum."""

def ask(question):
    hits = retrieve(question)
    if not hits:
        return "Not in my notes.", []

    context = "\n\n".join(f"[{h['source']}] {h['text']}"
                          for h, _ in hits)

    resp = client.chat.completions.create(
        model=MODEL, max_tokens=500, temperature=0,
        messages=[
            {"role": "system", "content": SYSTEM},
            {"role": "user", "content":
                f"<context>\n{context}\n</context>\n\nQuestion: {question}"},
        ],
    )
    return resp.choices[0].message.content, hits
```

Four rules, four failures

Each line closes something specific: ignoring the context, uncitable claims, confident guessing, rambling.

Why the tags?

They separate instructions from data. Without them, a note containing “ignore previous instructions” is a live prompt-injection hole. Retrieved text is untrusted input.

Test it like an engineer

1 The easy hit

Ask something stated almost word-for-word in your notes.

Pass: correct answer, right file cited.

If this fails, check what `retrieve()` returned before you touch the prompt.

2 The paraphrase

Ask the same thing using completely different vocabulary.

Pass: still finds it.

This is the test keyword search would fail.
It's the whole reason you used embeddings.

3 The trap

“Who won the 2011 Cricket World Cup?” — something the model knows but your notes don't contain.

Pass: `Not in my notes.`

Anyone can get a RAG app to answer. Getting it to **refuse** is the hard part — and it's the thing to demo when you show this to someone.

Give it a face

```
# app.py          streamlit run app.py
import streamlit as st

@st.cache_resource
def load_app():
    import rag
    return rag

rag = load_app()

st.title("Study Buddy")
q = st.text_input("Ask a question about your notes")

if q:
    answer, hits = rag.ask(q)
    st.markdown(answer)
    with st.expander("Sources"):
        for h, score in hits:
            st.markdown(f"**{h['source']}**  `{score:.2f}`")
            st.caption(h["text"][:300] + "...")
```

One gotcha

Streamlit re-runs the whole script on every keystroke. Without `@st.cache_resource` you re-embed everything, constantly.

Showing sources is not optional

It's the difference between “trust me” and “check me”. People forgive a wrong answer they can verify.

Deploy it free

GitHub, then Streamlit Community Cloud. Key goes in Secrets, never the repo. A live link beats a screenshot.

When the answers are wrong

SYMPTOM	MOST LIKELY CAUSE	WHAT TO CHANGE
Vague, waffly answers	Chunks too large	Drop to 120–180 words
Misses a fact that's clearly there	The fact was split across a chunk boundary	More overlap; split on headings first
Says “not in my notes” too often	<code>k</code> too small, or the floor too high	Try <code>k=5</code> , floor 0.2, and print the scores
Confidently invents things	Grounding rule too soft, or noise chunks got through	Harden the system prompt; add the floor
Right topic, wrong document	Semantic search ignores exact codes and IDs	Add keyword matching alongside
Good chunks, bad answer	Now it really is the prompt or the model	Tighten the prompt; try a bigger model

Print the retrieved chunks. Then print the exact final prompt. Only then blame the model.

Most people start at step three and lose a day.

Before you put this anywhere real

Free tiers

- Around 15 requests a minute, 1,500 a day
- Handle `429` with backoff — don't panic-rerun
- Put a `break` in your retry logic. Runaway loops, not real usage, are what burn quota
- Cache repeated questions

Keys

- Environment variables. Never in code, never in a repo
- `.gitignore` your `.env` before the first commit
- **Never call the API from browser JavaScript** — the key is public instantly. Proxy through your own backend
- Rotate a key the moment it's been anywhere public

Judgement

- **Free tiers are often free because your prompts may train the model.** Read the terms
- Never put someone else's personal or medical data through one
- Treat retrieved documents as untrusted input
- Show sources. Label AI output as AI output

If your data is genuinely sensitive, change `BASE_URL` to Ollama and run the whole thing offline. That's why we built it provider-agnostic.

Six things to build next

Study Buddy+

Today's app with PDFs, per-course filters, and quiz generation from your own notes.

Campus FAQ Bot

Answers from the student handbook, with citations, an escalation path, and a log of what it couldn't answer.

Resume ↔ JD Matcher

Score a CV against a job description section by section. Embeddings only — no LLM in the core loop.

Notes → Tasks API

Lab 2 wrapped in FastAPI. POST messy notes, get validated JSON back. A real microservice.

Codebase Explainer

RAG over a repository, chunked by function. “Where is auth handled?”

Paper Digest

Ten PDFs in, structured summaries out, then ask questions across all of them at once.

Full briefs — build steps, stretch goals and how to demo each one — are in the handout.

This week

- 1 **Tonight, 20 minutes.** Point the app at a folder that matters to you. Ask it ten real questions. Write down every wrong answer.
- 2 **Tomorrow, an hour.** Turn those ten into a CSV with the answers you expected. Change one thing — chunk size — re-run, and watch the score move.
- 3 **This weekend.** Add the UI and deploy it.
- 4 **Next week.** Pick one project and give it two evenings.

Everyone here can now build a demo. Almost nobody can say whether their demo got *better* after a change.

The day you can say “**28 out of 30, up from 24 after I fixed the chunking**” is the day you stop sounding like a student.

Keep a `prompts.md` in every AI project. Each time you fix a prompt, paste the before and after and one line on why. In three months it'll be the most valuable file in the repo.

You didn't learn an API. You learned the shape of AI applications.

Retrieve the right context. Instruct precisely. Demand a format you can verify. Measure whether it worked.

Every model, every provider and every framework you meet from here is a variation on those four moves.

Questions